The creators of the `DeltaTorchLoader` performed a few benchmarks with Petastorm. The benchmark was shared at the **Data & AI Summit** (**DAIS**) and is featured in *Figure 3.31*. In this project, we compare Petastorm to the classic Torch loader in *Chapter 6* to show the performance gain. The comparison demonstrates an incredible speed increase when reading the batch of data. We've also included a great video on `TorchDeltaLoader` in *Further reading* if you want to learn more.

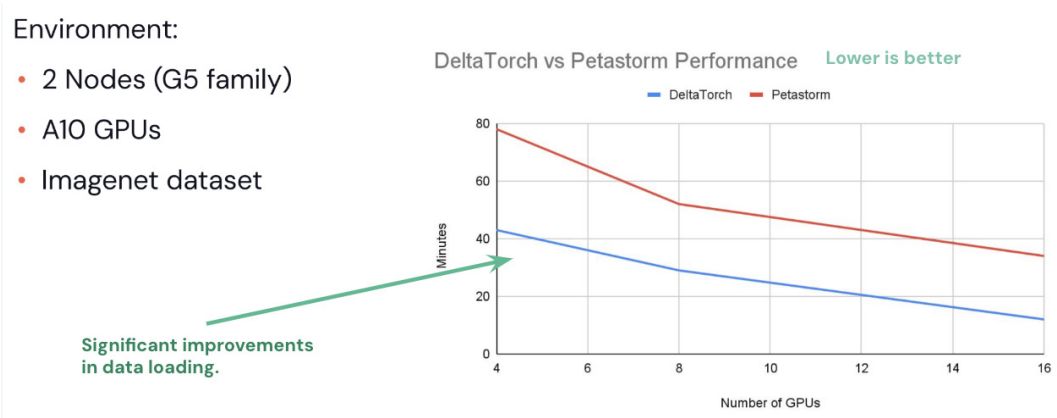


Figure 3.31 – DeltaTorchLoader benchmark - Performance comparison between DeltaTorch and Petastorm loaders

An alternative to using `DeltaTorchLoader`, is to keep file names in your Delta table instead of the images and collect them while passing them to the main PyTorch Loader with the trainer function. Keeping files in Delta is essential to avoid duplicates and control the list used during the training and validation, as you can pass the Delta version to the MLFlow during the tracking for full replication purposes.

Optimizing Our Data

Once our tables are created and written to the storage, we use a few functions to improve read performance on the Delta tables. We use the SQL magic command, `%sql`, to perform these operations using SQL in the Python notebook:

1. We use `OPTIMIZE` to keep an ideal number of files (*Figure 3.32*).

```
%sql
-- you can set up a widget to a notebook and consume widgets via $ variables with SQL
--OPTIMIZE delta.`/Volumes/$catalog/$database_name/files/intel_image_clf/valid_imgs_main.
delta`
OPTIMIZE delta.`/Volumes/ml_in_action/cv_clf/files/intel_image_clf/train_imgs_main.delta`
```

Figure 3.32 – Optimizing the file size and count of the training table